



Chapter – 1

Introduction to XML





Before we continue with the XML, we have to revise certain fact about the terms like HTML and JavaScript.

HTML

It stands for **HyperText Markup Language**. It is used to design web pages using a markup language. HTML is the combination of Hypertext and Markup language. Hypertext defines the link between the web pages. A **markup language** is used to define the text document within tag which defines the structure of web pages. This language is used to annotate (make notes for the computer) text so that a machine can understand it and manipulate text accordingly. Most markup languages (e.g. HTML) are human-readable. The language uses tags to define what manipulation has to be done on the text.

HTML is a markup language used by the browser to manipulate text, images, and other content, in order to display it in the required format. HTML was created by Tim Berners-Lee in 1991. The first-ever version of HTML was HTML 1.0, but the first standard version was HTML 2.0, published in 1999.

Elements, tags, and attributes are basic concepts in HTML.

HTML **element** is a main structural unit of a web page. HTML tags are used to define HTML elements, and attributes provide additional information about these elements.

HTML **tags** are used to structure website content (text, hyperlinks, images, media, etc). Tags are not displayed in the browsers, they only “instruct” browsers how to show the content of the web page. There are over 100 tags in HTML, and can be used for styling the web content like text, images, videos, audios etc. HTML tags are written in angle brackets (e.g <html>).

HTML **attributes** are added to an HTML element to provide additional information about it. For example, to define an image with tag, we can use attributes like: src, height, width attributes to provide information about its source, height, width correspondingly.

JavaScript

JavaScript is a **scripting or a programming language**, allowing developers to perform complex features on web pages. Initially, this language was created for making web pages alive. In JavaScript, the programs are called scripts. One can write them in the HTML of a web page, then it will automatically run once the page loads. At this point, JavaScript is completely different from another language, called Java.

Initially, when JavaScript was created, it was called “**LiveScript**”. Then, as Java was extremely popular in that period, then it was decided to call it JavaScript to position it relative to Java. But, over the years, JavaScript has transformed into a completely independent language, with its specification, known as **ECMAScript**, having no relation to Java. Modern JavaScript can both work in the browser and on the server. Basically, it can run on any device that has a specific program known as the JavaScript engine.



What In-browser JavaScript Does

Modern JavaScript is considered a safe programming language. It never provides low-level access to the memory or CPU as it was made for the browsers that don't require it. The capabilities of this language highly rely on the environment it runs in. For example, Node.js includes functions allowing JavaScript to write and read arbitrary files, implement network requests, and so on. In-browser JavaScript does anything related to web page manipulation.

For example, with in-browser JavaScript, you can do the following:

1. Adding new HTML to the page, changing the content, modifying the styles.
2. Reacting to user actions, running on mouse clicks, key presses, and more.
3. Sending requests over the network to remote servers.
4. Getting and setting cookies, asking questions to visitors, sending messages.
5. Remembering the data on the client-side.

Limitations of In-browser JavaScript

The capabilities of JavaScript are limited for the purpose of keeping the user's safety. With it, an evil web page can't access private information or harm the user's data.

Here are some examples of such restrictions:

- JavaScript has no direct access to OS functions. It can't read and write arbitrary files on the hard disc, copy or execute them.
- Different windows/tabs don't recognize each other. JavaScript from one page is not able to access the other one, in case they are from different sites. It's known as "Same Origin Policy".
- JavaScript allows communication over the net to the server from where the page comes from. But, its capability of receiving data from the other site is prohibited. That's a safety limitation.

What Makes JavaScript Specific

There are at least three perfect things about JavaScript:

1. It supports complete integration with CSS and HTML.
2. It provides straightforward ways of doing simple things.
3. It is supported by almost all the major browsers and is performed by default.

It is the exclusive browser technology that encompasses the three great things above.

In modern programming, JavaScript is the most widespread tool that helps to create browser interfaces. It also allows creating mobile applications, servers, and much more. Our JavaScript book consists of several sections that cover all the information you need to learn this unique programming language. Each of the chapters includes both the theory and practical cases to make it easier for beginners to grasp the language.



XML (eXtensible Markup Language)

XML is an independent tool for storing and transporting data other than the hardware and the software. It doesn't depend on the platform and the software (programming language). You can write a program in any language on any platform (Operating System) to send, receive or store data using XML. XML is a markup language for documents containing structured information.

Structured information contains both content (words, pictures, etc.) and some indication of what role that content plays (for example, content in a section heading has a different –meaning from content in a footnote, which means something different than content in a figure caption or content in a database table, etc.). Almost all documents have some structure.

A markup language is a mechanism to identify structures in a document. The XML specification defines a standard way to add markup to documents.

Why XML?

Platform Independent and Language Independent: The main benefit of xml is that you can use it to take data from a program like Microsoft SQL, convert it into XML then share that XML with other programs and platforms. You can communicate between two platforms which are generally very difficult.

The main thing which makes XML truly powerful is its international acceptance. Many corporation use XML interfaces for databases, programming, office application mobile phones and more. It is due to its platform independent feature.

1.1 Characteristics and Use of XML

XML was designed for describing well-formed data. Also XML have some strict rules. That rules follow every XML document. Following are some characteristics of the XML.

Characteristics of XML

- **XML is extensible**

Unlike HTML, XML applications allows to have user defined tags in structured manner. User is able to add or remove the tags in the defined structure. Older one with predefined structure as well updated one, both will work perfectly.

example1.1	example1.1(updated)
<pre><?xml version="1.0" encoding="utf-8"?> <bca> <subject>405 : Web Design-2</subject> <credit>4</credit> <hoursperweek>4 Hrs</hoursperweek> <body>Web Design requires designers to create graphics, typography as well as images which are used only on the World Wide Web. While creating any design, web designers need to maintain balance between creating a good</pre>	<pre><?xml version="1.0" encoding="utf-8"?> <bca> <subject>405 : Web Design-2</subject> <credit>4</credit> <hoursperweek>4 Hrs</hoursperweek> <objective>To make students aware of web terminology and website designing tools. Student can understand and implement the real functions of website development. </objective></pre>



design as well as the speed and efficiency for the webpage/website. </body> </bca>	<body>Web Design requires designers to create graphics, typography as well as images which are used only on the World Wide Web. While creating any design, web designers need to maintain balance between creating a good design as well as the speed and efficiency for the webpage/website. </body> </bca>
---	---

- **XML focuses on data rather than how it looks**

One of the reason, XML is popular because it focuses on data rather than data presentation. The other markup language such as HTML is used for data presentation. This separates the data and its presentation part and gives us the freedom to present the data, the way we want, once we receive it using XML.

Two or more systems can receive the same data from a same XML and present it in a different way using other markup language such as HTML.

- **XML is public standard**

XML was developed by W3C (World Wide Web Consortium) organization as an open standard under February 1998.

- **XML is easy and efficient for data sharing**

Since XML is software and hardware independent, it is easier to share data between different systems with different hardware and software configuration. Any system with any programming language can read and process a XML document.

- **XML is compatible with other markup languages like HTML**

It is so much easier to read the data from XML and display it on an GUI (graphical user interface) using HTML markup language.

When the data changes over time, we need not to make any changes in the HTML.

- **XML supports platform transition**

The main reason why changing to new systems and platform is challenging, because it involves the headache of data conversion between incompatible formats which often results in data loss. XML simplifies this process as the data is transported on new upgraded systems without any data loss.

- **XML supports validation**

A XML document can be validated using DTD or XML schema. This ensures that the XML document is syntactically correct and avoids any issues that may arise due to the incorrect XML.

- **XML supports Unicode**

XML supports Unicode that allows it to communicate almost any information in any written human language.



- **XML adopts technology advancements**

The reason why XML is popular and being used from a very long time is because, it can adapt to the new technologies because of its platform-independent nature.

Uses of XML

XML is playing a vital role in the web. XML has a variety of uses in the sectors like Web, e-business, and mobile applications. Few XML-based Languages include XHTML, RSS, SMIL, WSDL, WAP, and SOAP. XML files are used to develop database-driven types. Due to their Flexibility, they could transfer data without missing descriptive information among corporate databases. Few examples like, with XML business-to-business applications share information electronically between buyers and sellers. It's an excellent choice for exchange formats with small collections of data.

It can be described as follows:

- XML can work behind the scene to simplify the creation of HTML documents for large web sites.
- XML can be used to exchange the information between organizations and systems.
- XML can be used for offloading and reloading of databases.
- XML can be used to store and arrange the data, which can customize your data handling needs.
- XML can easily be merged with style sheets to create almost any desired output.
- Virtually, any type of data can be expressed as an XML document.

1.2 XML syntax (Declaration, tags, elements)

XML syntax rule is used while writing an XML document or an XML application. It is a very simple and straight forward to learn and code.

Check out the below sample code for understanding of the XML syntax rules:

Sample Code (example 1.2)

```
<?xml version="1.0" encoding="UTF-8"?>
<assign>
  <to>SYBCA</to>
  <from>Faculty</from>
  <subject>405 : WebDesign-1</subject>
  <info>Each Assignment of Subject carries 30 marks. </info>
</assign>
```

XML Declaration / Prolog rules

<?xml version="1.0" encoding="UTF-8"?>

- This line is called **XML Prolog** or **XML declaration**.
- This line is optional i.e., it can be either used or not in an XML document. However, it should be the very first line if used.



- The version="1.0" is the version of the XML currently used. There are various versions of XML available.
- The encoding="UTF-8" specifies the character encoding used while writing an XML document, for example, èëé is for French and so on. Its default value is "UTF-8".
- This declaration is case sensitive for example "xml" should must be in lower case in.

XML Root element rules

Every XML files should have one or more Root elements to avoid error.

- In the example 1.2 stated above the Root element is <assign> and all the remaining elements <to>, <from>,<subject>,<info> are the child elements and reside within the root element <assign>.
- It is case sensitive.

XML Element rules

- The XML elements should have a closing element for example **<info category = "internal">Sample code</info>** is correct but **<info category = "internal">Sample code** is not correct because it does not contain the closing element and it will throw an error and vice-versa.
- The elements in XML should be nested properly otherwise it will throw an error. For example **<to> <from> VBP </from> </to>** is nested correctly but **<to> <from> VBP </to> </from>** is wrong because if <from> is opened inside the <to> element then this should also end inside of the </to> element.
- It is also case sensitive i.e., the starting and closing element should be in the same case. For example **<to>....</to>** is correct but **<to>.....</To>** is not correct and it will throw an error.

XML Attribute rules

- The XML attribute is having two part one is Name and other is its value. It resides inside of the opening of an XML element. For example: **<info category = "internal"> Each Assignment of Subject carries 30 marks.</info>** Here category is the attribute name and internal is its value and the attribute value should either be in a single quotation or in double quotation otherwise it will throw an error. The Attribute Name is written without any quotation.
- The XML attribute is also case sensitive.
- An XML element can have multiple attributes but cannot have the same attribute names in the same element.
- For example: **<info category="internal" sem="IV"> Each Assignment of Subject carries 30 marks. </info>**



Above attributes is correct because of having multiple attributes with the different attribute name.

- `<info category="internal" category="IV">` Each Assignment of Subject carries 30 marks. `</info>`

Above attribute is wrong because of having the same attribute name in a single element.

XML Tags

XML tags are the important features of XML document. It is similar to HTML but XML is more flexible than HTML. It allows to create new tags (user defined tags). The first element of XML document is called root element. The simple XML document contains opening tag and closing tag. The XML tags are case sensitive i.e. `<root>` and `<Root>` both tags are different. The XML tags are used to define the scope of elements in XML document.

XML Tags Property Rules

- Every XML document must have a root tag which encloses the XML document. It is not necessary to name of root tag is root. The name of root tag is any possible tag name.
- The XML document must have start-tag, so first starting tag is known as root tag. The opening tag started with `<` bracket followed by tag name or element name and close with `>` bracket.
- The tag which is started by start tag must end with the same tag with forward slash (end tag), or in other words every XML document must be ended with end-tag. The end tag started with `<` followed by `/` and its pair tag name ended with `>`
- In XML, tags are case sensitive. It means that `<Root>` and `<root>` both are different tags.
- The tag which contains no content are known as empty tags.
- XML tag must be closed in appropriate order. For example, an XML tag opened inside another element must be closed before the outer element is closed.

1.3 Root Element, Case Sensitivity

XML document must have a root element. A root element can have child elements and sub-child elements. We already have discussed it in detail on page. 7(Previous page).

Some basic rules of XML are mentioned as follows:

- **XML is Case Sensitive**

While defining the tags in XML we have to keep in mind that the tags which we defined must be strictly case sensitive. If a single letter in the tag name differs in cases for starting and ending it will be considered as an invalid XML declaration.



- **XML Comments**

XML comments are just like HTML comments. We know that the comments are used to make codes more understandable other developers.

XML Comments add notes or lines for understanding the purpose of an XML code. Although XML is known as self-describing data but sometimes XML comments are necessary.

Rules for adding XML comments

- Don't use a comment before an XML declaration.
- You can use a comment anywhere in XML document except within attribute value.
- Don't nest a comment inside the other comment.

- **No overlapping for elements in XML**

All the elements in XML should be properly nested and they should not overlap.

- **White-spaces are preserved in XML**

Unlike HTML that doesn't preserve white space, the XML document preserves white spaces.

1.4 XML Document

XML Document forms the full structure of xml formatted data composed with prolog and root element nested with other elements. There would be only one root element, where root element encloses many other elements inside the final inner element holds the data. XML Document can be divided as three components.

They are:

1. Prolog
2. Elements (Root or Other)
3. Data

Below is the example of Students XML document with the root element "students".

students.xml (example 1.3)

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<?xml-stylesheet type="text/css" href="/style/design"?>
<!-- This is a comment -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0
Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<students>

    <stud>

        <name> Amit</name>

        <city> Surat</city>
```



```

</stud>

<stud>

    <name> Manish</name>

    <city> Bharuch</city>

</stud>

</students>

```

In the above example 1.3,

Document Prolog —

```

<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<?xml-stylesheet type="text/css" href="/style/design"?>
<!DOCTYPE      html      PUBLIC      "-//W3C//DTD      XHTML      1.0
Strict//EN""http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

```

Root Element — <students>

Other Elements — <stud> , <name> , <city>

Data — Amit, Surat, Manish, Bharuch

Let's See above concepts in descriptive manner.

1.4.1 XML Prolog section

XML Prolog is the component added in the beginning of an XML document. Otherwise, we can say that whatever it appears before the document's root element can be considered as Prolog. XML Prolog includes XML declaration, DOCTYPE and comments, processing instructions too. (See example 1.3)

DOCTYPE is used to mention the Document Type Definition and it is used for validation purpose. We could call it as a valid XML if it get validated through DTD. DTD will consist the structure of an XML document. DTD defines the elements to be used in the XML document. (See example 1.3)

```

<!DOCTYPE students
[
<!ELEMENT students (stud)>
<!ELEMENT stud (name,city)>
<!ELEMENT name (#DATA)>
<!ELEMENT city (#DATA)>
]>

```

1.4.2 XML Element section (Root, Other elements)

XML Element is the main component which holds the data from start tag till end tag. It is used as a container to store text elements, attributes, media objects etc or in case of empty elements it is delimited by an empty tag. Every XML documents contain at least one element whose scopes are delimited by start and end tags. Every XML document can have only one root element. In the above shown example 1.3 there is one root element listed as <students>. And the root element is able to hold other sub



elements. XML document consists one or more elements enclosed in root element. Elements naming represent the self-explanatory form of data inside it. Elements contains attributes which are used to specify additional information about the element. In the above shown example 3, inner elements are listed as <stud>, <name>, <city>.

Syntax

<element-name attributes> Contents...</element-name>

In the syntax above the attributes are used to define the XML element property and these attributes are separated by white space. It associates the name with a value, which is a string of characters.

Empty Elements

An element in XML document which does not contains the content is known as Empty Element. The basic syntax of empty element in XML as follows:

Syntax

<element-name attributes></element-name>

OR

<element-name attributes />

Rules of XML elements

There are some rules to create XML elements which are given below:

- An element can contain alphanumeric values or characters. But only three special characters are required in the names these are hyphen, underscore and period.
- Names are case sensitive. It means lower case letters have different meaning and upper case characters have different meaning. For example city, City, CITY are different names.
- Both start and end tags for elements need to be same.
- An element, which is a container, can contain text or elements

Data

An XML element hold the value which is considered as data value.

1.5 XML Declaration and Rules of Declaration

XML documents can contain an XML declaration that if present, must be the first construct in the document. An XML declaration is made up of as many as three name/value pairs, syntactically identical to attributes. The three attributes are a mandatory version attribute and optional encoding and standalone attributes. The order of these attributes within an XML declaration is fixed.



Syntax

```
<?xml
  version = "version_number"
  encoding = "encoding_declaration"
  standalone = "standalone_status"
?>
```

The XML declaration begins with the character sequence **<?xml** and ends with the character sequence **?>**. Note that although this syntax is identical to that for processing instructions, the XML declaration is not considered to be a processing instruction.

All XML declarations have a **version** attribute with a value that must be **1.0**

The **character encoding** used for the document content can be specified through the encoding attribute. XML documents are inherently Unicode, even when stored in a non-Unicode character encoding. The XML recommendation defines several possible values for the encoding attribute. For example, UTF-8, UTF-16, ISO-10646-UCS-2, and ISO-10646-UCS-4 all refer to unicode/ISO-10646 encodings, whereas ISO-8859-1 and ISO-8859-2 refer to 8-bit Latin character encodings. Encodings for other character sets including Chinese, Japanese, and Korean characters are also supported. It is recommended that encodings be referred to using the encoding names registered with the Internet Assigned Numbers Authority (IANA). All XML processors are required to be able to process documents encoded using UTF-8 or UTF-16, with or without an XML declaration. The encoding of UTF-8 and UTF-16 encoded documents is detected using the Unicode byte-order-mark. The XML declaration is mandatory if the encoding of the document is anything other than UTF-8 or UTF-16. In practice, this means that documents encoded using US-ASCII can also omit the XML declaration because US-ASCII overlaps entirely with UTF-8.

Only one encoding can be used for an entire XML document. It is not possible to “redefine” the encoding part of the way through. If data in different encodings need to be represented, then external entities should be used.

If an XML document can be read with no reference to external sources, it is said to be a stand-alone document. Such documents can be annotated with a **standalone** attribute with a value of **yes** in the XML declaration. If an XML document requires external sources to be resolved to parse correctly and/or to construct the entire data tree (for example, a document with references to external general entities), then it is not a stand-alone document. Such documents may be marked **standalone='no'**, but because this is the default, such an annotation rarely appears in XML documents.

Rules of XML Declaration

- An XML declaration should abide with the following rules –



- If the XML declaration is present in the XML, it must be placed as the first line in the XML document.
- If the XML declaration is included, it must contain version number attribute.
- The Parameter names and values are case-sensitive.
- The names are always in lower case.
- The order of placing the parameters is important. The correct order is: version, encoding and standalone.
- Either single or double quotes may be used.
- The XML declaration has no closing tag i.e. `</?xml>`

